

Data Analytics Quick Recap

1. NumPy Fundamentals

- Array Creation:
`np.arange(10,51), np.random.randint(1,100, size=(3,3))`
- Operations:
 - Statistics: `arr.mean(), np.median(arr), arr.std()`
 - Conditional: `arr[arr % 2 == 1] = -1`
 - Reshaping: `matrix.reshape(1,9)`
- Boolean Indexing:
 - Filter: `arr[arr > 25]`
 - Count: `np.count_nonzero(arr % 5 == 0)`

Code & example :

```
import numpy as np

# 1D array from 10 to 50
arr1 = np.arange(10, 51)

# 3x3 matrix with random integers
matrix = np.random.randint(1, 101, size=(3,3))

# Statistical operations
mean_val = np.mean(arr1)
median_val = np.median(arr1)
std_val = np.std(arr1)

# Replace odd numbers with -1
arr1[arr1 % 2 == 1] = -1

# Reshape matrix
reshaped = matrix.reshape(1, 9)

# Boolean indexing
gt_25 = matrix[matrix > 25]
div_by_5 = np.count_nonzero(matrix % 5 == 0)

# Random array operations
rand_arr = np.random.randint(1, 101, 20)
mat_5x5 = np.arange(1, 26).reshape(5,5)
```

```
mat_4x5 = rand_arr.reshape(4,5)
cols = mat_4x5[:, [1,3]] # 2nd and 4th columns

# Conditional replacement
mat_5x5[mat_5x5 % 5 == 0] = -1
```

2. Pandas Workflows

- Data Structures:
 - *Series*: `pd.Series([87,45,...], index=['A','B',...])`
 - *DataFrame*: `pd.DataFrame({'Name': [...], 'Math': [...], ...})`
- Operations:
 - New Column: `df['Average'] = df[['Math','Science']].mean(axis=1)`
 - Sort: `df.sort_values('Average', ascending=False)`
 - Filter: `df[(df.Math>70) & (df.Science>70)]`
- Missing Data:
 - Fill NaNs: `df.fillna(df.mean())`
 - Drop NaNs: `df.dropna()`
 - Grouping: `df.groupby('species')['sepal_length'].mean()`

Code & Example :

```
import pandas as pd

# Series creation
scores = pd.Series([87, 45, 92, 67, 55],
                   index=['A','B','C','D','E'])

# DataFrame creation
students = pd.DataFrame({
    'Name': ['Alice','Bob','Charlie','David','Eva'],
    'Math': [87,45,92,67,55],
    'Science': [78,66,80,72,60]
})

# New column and filtering
students['Average'] = students[['Math','Science']].mean(axis=1)
sorted_students = students.sort_values('Average', ascending=False)
top_students = students[(students.Math>70) & (students.Science>70)]
```

```

# Missing data handling
students.loc[5] = ['Frank', np.nan, np.nan, np.nan]
students.fillna(students.mean(numeric_only=True), inplace=True)
students.dropna(inplace=True)

# Iris dataset operations
iris = pd.read_csv('iris.csv')
print(iris.head())
print(iris.info())
print(iris.describe())

# Filtering and grouping
subset = iris[['sepal_length', 'species']]
filtered = iris[(iris.petal_width>1.0) & (iris.species=='setosa')]
species_avg = iris.groupby('species')['sepal_length'].mean()
species_count = iris['species'].value_counts()

# Missing value handling
iris.loc[:2, 'sepal_length'] = np.nan # Inject missing values
iris['sepal_length'].fillna(iris['sepal_length'].mean(), inplace=True)
iris.dropna(subset=['species'], inplace=True)

# Sorting and analysis
iris.sort_values('petal_length', ascending=False, inplace=True)
top_species = iris['species'].value_counts().nlargest(2)
import pandas as pd

# Series creation
scores = pd.Series([87, 45, 92, 67, 55],
                    index=['A', 'B', 'C', 'D', 'E'])

# DataFrame creation
students = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eva'],
    'Math': [87, 45, 92, 67, 55],
    'Science': [78, 66, 80, 72, 60]
})

# New column and filtering
students['Average'] = students[['Math', 'Science']].mean(axis=1)
sorted_students = students.sort_values('Average', ascending=False)
top_students = students[(students.Math>70) & (students.Science>70)]

# Missing data handling
students.loc[5] = ['Frank', np.nan, np.nan, np.nan]
students.fillna(students.mean(numeric_only=True), inplace=True)
students.dropna(inplace=True)

# Iris dataset operations
iris = pd.read_csv('iris.csv')
print(iris.head())
print(iris.info())
print(iris.describe())

```

```
# Filtering and grouping
subset = iris[['sepal_length', 'species']]
filtered = iris[(iris.petal_width>1.0) & (iris.species=='setosa')]
species_avg = iris.groupby('species')['sepal_length'].mean()
species_count = iris['species'].value_counts()

# Missing value handling
iris.loc[:, 'sepal_length'] = np.nan # Inject missing values
iris['sepal_length'].fillna(iris['sepal_length'].mean(), inplace=True)
iris.dropna(subset=['species'], inplace=True)

# Sorting and analysis
iris.sort_values('petal_length', ascending=False, inplace=True)
top_species = iris['species'].value_counts().nlargest(2)
```

3. Matplotlib Visualization

- Basic Plots:

python

```
plt.plot(x, x**2, 'b-', label="y=x²") # Line plot
```

```
plt.bar(countries, population, hatch='//') # Bar chart
```

```
plt.hist(np.random.randn(1000), bins=20) # Histogram
```

- Advanced Features:
 - Subplots: `plt.subplot(1,2,1) + plt.subplot(1,2,2)`
 - Scatter: `plt.scatter(x,y, c=distance, cmap='viridis')`
 - Colorbar: `plt.colorbar()`
- Customization:

Always include:

```
plt.title(), plt.xlabel(), plt.ylabel(), plt.grid(), plt.legend()
```

Code & Example :

```
import matplotlib.pyplot as plt
import seaborn as sns

# Line plots
x = np.arange(0, 11)
plt.plot(x, x**2, 'b-', label='y=x²')
plt.title('Quadratic Plot')
plt.xlabel('X-axis'); plt.ylabel('Y-axis'); plt.grid()

# Trigonometric functions
```

```

x = np.linspace(0, 2*np.pi, 100)
plt.plot(x, np.sin(x), 'r--', label='sin(x)')
plt.plot(x, np.cos(x), 'g:', label='cos(x)')
plt.legend(); plt.grid()

# Subplots
plt.subplot(1,2,1)
plt.plot(x, np.sin(x), 'b')
plt.title('Sine Wave')
plt.subplot(1,2,2)
plt.plot(x, np.cos(x), 'r')
plt.title('Cosine Wave')
plt.tight_layout()

# Bar chart
countries = ['USA', 'India', 'Brazil', 'Germany', 'Japan']
population = [331, 1388, 213, 83, 126]
plt.bar(countries, population, hatch=['/', 'x', '+', 'O', '*'])
for i, val in enumerate(population):
    plt.text(i, val, str(val), ha='center', va='bottom')
plt.ylabel('Population (millions)')

# Histogram
data = np.random.randn(1000)
plt.hist(data, bins=20, alpha=0.7)
plt.grid(True)

# Scatter plot
x = np.random.rand(100)
y = np.random.rand(100)
dist = np.sqrt(x**2 + y**2)
plt.scatter(x, y, c=dist, cmap='viridis')
plt.colorbar(label='Distance from Origin')

# Iris visualizations
species_avg.plot(kind='line', title='Average Sepal Length')
iris['species'].value_counts().plot(kind='bar')
plt.scatter(iris['sepal_length'], iris['petal_length'],
            c=pd.factorize(iris['species'])[0], cmap='viridis')
plt.hist(iris['sepal_width'], bins=10)

# Advanced plots
sns.boxplot(x='species', y='sepal_length', data=iris)
corr = iris.select_dtypes(include=np.number).corr()
sns.heatmap(corr, annot=True)

# Export data
iris.to_csv('cleaned_iris.csv', index=False)

```

Key Integration Points:

1. **Data Flow:**

- NumPy arrays → Pandas DataFrames → Matplotlib visualizations
- Example: `plt.plot(numpy_array)` or `df.plot()`

2. **Common Patterns:**

python

Data preparation

`data = np.random.randn(1000)`

`df = pd.DataFrame(data, columns=['values'])`

Visualization

`plt.hist(df['values'], bins=20)`

`plt.title('Distribution')`

3. **Pro Tips:**

- Use `pd.factorize()` to convert categorical data to numeric for coloring
- Always call `plt.tight_layout()` before saving figures
- Use `numeric_only=True` in Pandas operations with mixed data types
- For large datasets, use `sns.kdeplot()` instead of histograms